



THE NATIONAL CERT OF **PAKISTAN**

L-Block, Pak Secretariat, Islamabad



Submitted By	Hafiz Muhammad Moawiz
Degree	BS - CY
Domain	Software Security Testing
Submitted To	Team NCERT

Web Application Security — Foundations, Reconnaissance & Automated Scanning

Week 1 Task Report (16-7-2026)

As a cybersecurity student, Kali Linux is already used this semester as the main working environment for coursework. Because of this, most of the tools required for this task — DVWA, OWASP Juice Shop, OWASP ZAP, Nikto, and WhatWeb — were already set up on it from earlier coursework. The only tool set up separately, specifically for this task, was Wappalyzer, which was added as a browser extension.

Targets Overview

#	Target URL	Target Name	Status
1	http://localhost/DVWA/	DVWA	Completed (2 – 8)
2	http://localhost:3000	OWASP Juice Shop	Completed (9 – 14)

Introduction

This report covers the Week 1 task of the internship. The goal of Week 1 was to build a small, safe testing lab and practice the first steps of web security testing.

Testing Approach

A Black Box style of testing was used. This means the tester acted like a normal outside visitor with no special access, and used tools to explore the site the same way a real attacker would look at it from outside.

Tools Used

Tool	What it does
Wappalyzer	A browser extension that looks at a website and tells us what technology (server, language, frameworks) it is built with.
WhatWeb	A command-line tool that also detects website technology, but looks deeper — it checks response headers, cookies and page titles.
OWASP ZAP	The main testing tool. It was used to explore each site by hand, map out all pages and links (Spider / AJAX Spider), and run an automatic scan that tries common attacks (Active Scan).
Nikto	A tool that scans the web server itself for outdated software, wrongly exposed folders, and missing security settings.

The same 5 steps were followed for both targets, in this order:

- 1. Wappalyzer — find the technology used on the site.
- 2. WhatWeb — double-check the technology from the command line.
- 3. ZAP Manual Explore + Spider + AJAX Spider — click through the site and let ZAP map out every page, folder and API endpoint.
- 4. Nikto — run an automatic scan of the web server for known weaknesses.
- 5. ZAP Active Scan — run an automatic scan that actually tries safe attack payloads (like XSS or path traversal) on every page and form.

TARGET 1

DVWA — Damn Vulnerable Web Application

2. Target Information

Target	http://localhost/DVWA/
Web Server	Apache/2.4.65 (Debian)
Backend Language	PHP
Application Type	Traditional PHP website with login page (DVWA)
Testing Type	Black Box, followed by an authenticated scan using DVWA's own login (admin account)

3. Reconnaissance

3.1 Technology Check (Wappalyzer)

Wappalyzer is a browser extension. It checks a page in the browser and reports what technology it can see. On DVWA, it only reported the Apache web server.

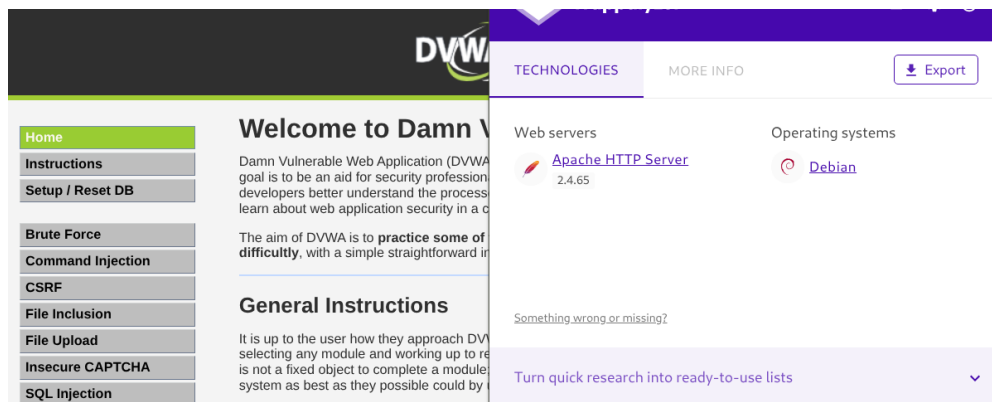


Figure 1 — Wappalyzer result for DVWA: only Apache / HTTP server detected

This is a short result because Wappalyzer mostly looks for things visible from the browser side.

3.2 Technology Check (WhatWeb)

WhatWeb was run from the terminal to double check and dig a bit deeper (it also reads cookies, headers, and page titles).

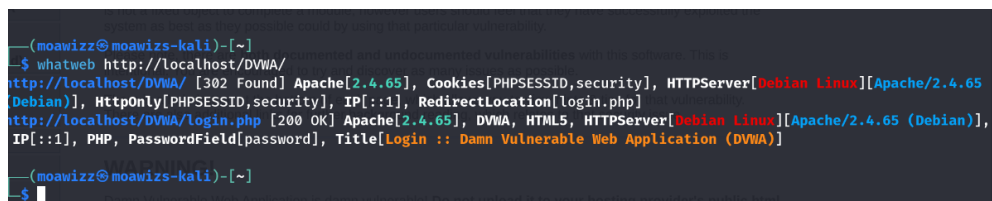
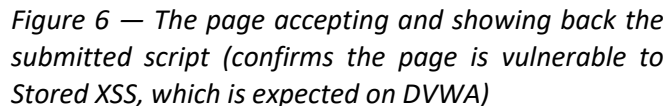


Figure 2 — WhatWeb result: Apache 2.4.65 on Debian Linux, PHP, DVWA cookies (PHPSESSID, security)

This confirmed the server is Apache 2.4.65 running on Debian Linux, the backend language is PHP, and the site sets two cookies: PHPSESSID (a normal session cookie) and security (DVWA's own cookie that remembers the chosen difficulty level).

3.3 Manual Exploring

Every page of DVWA was opened by hand in a ZAP-connected browser, so ZAP could record every link and form. While exploring, a few basic attack attempts were also tried on purpose, as practice, to see how the pages respond:



4. Automated Scanning

After mapping the site, two automatic scanning tools were used to look for known weaknesses.

4.1 Server Scan (Nikto)

Nikto checks the web server itself for outdated software, exposed folders, and missing security headers.

```
(root@kali:~/dvwa-2.0.1) [~]
$ nikto -h http://localhost/DVWA/
Nikto v2.5.0
-----
Target IP: 127.0.0.1
Target Hostname: localhost
Target Port: 80
Start Time: 2026-07-16 18:39:14 (GMTS)
-----
Server: Apache/2.4.65 (Debian)
/DVWA/: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
/DVWA/: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
Root page /DVWA redirects to: login.php
No CGI Directories found (use '-C all' to force check all possible dirs)
OPTIONS: Allowed HTTP Methods: OPTIONS, HEAD, GET, POST .
/DVWA/config/: Directory indexing found.
/DVWA/config/: Configuration information may be available remotely.
/DVWA/tests/: Directory indexing found.
/DVWA/tests/: This might be interesting.
/DVWA/database/: Directory indexing found.
/DVWA/database/: Database directory found.
/DVWA/docs/: Directory indexing found.
/DVWA/login.php: Admin login page/section found.
/DVWA/.git/index: Git Index file may contain directory listing information.
/DVWA/.git/HEAD: Git HEAD file found. Full repo details may be present.
/DVWA/.git/config: Git config file found. Infos about repo details may be present.
/DVWA/.gitignore: gitignore file found. It is possible to grasp the directory structure.
/DVWA/.dockerignore: .dockerignore file found. It may be possible to grasp the directory structure and learn more about the site.
7850 requests: 0 error(s) and 16 item(s) reported on remote host
End Time: 2026-07-16 18:39:27 (GMTS) (13 seconds)
-----
1 host(s) tested
```

Figure 9 — Nikto scan result: 16 issues reported, including exposed .git, config, database, tests and docs folders

Nikto found that several folders inside DVWA can be opened directly and show a list of their files (this is called "directory indexing"). Three of these were checked by hand to confirm they were real and not a false alarm:

Index of /DVWA/tests

Name	Last modified	Size	Description
Parent Directory		-	
README.md	2026-07-09 22:08	321	
test_url.py	2026-07-09 22:08	3.6K	

Apache/2.4.65 (Debian) Server at localhost Port 80

Figure 10 — Confirmed: the /tests/ folder is open lists its files

Index of /DVWA/database

Name	Last modified	Size	Description
Parent Directory		-	
bac_setup.sql	2026-07-09 22:08	1.0K	
create_mssql_db.sql	2026-07-09 22:08	1.1K	
create_oracle_db.sql	2026-07-09 22:08	1.1K	
create_postgresql_db.sql	2026-07-09 22:08	810	
create_sqlite_db.sql	2026-07-09 22:08	945	

11 — Confirmed: the /database/ folder is open and lists its files Figure

Index of /DVWA/.git

Name	Last modified	Size	Description
Parent Directory		-	
HEAD	2026-07-09 22:08	23	
config	2026-07-09 22:08	262	
description	2026-07-09 22:08	73	
hooks/	2026-07-09 22:08	-	
index	2026-07-09 22:08	27K	
info/	2026-07-09 22:08	-	
logs/	2026-07-09 22:08	-	
objects/	2026-07-09 22:08	-	
packed-refs	2026-07-09 22:08	837	
refs/	2026-07-09 22:08	-	

Figure 12 — Confirmed: the /.git/ folder is open. This is the most serious of the three, because a .git folder can contain the site's full source code history

4.2 Automated Vulnerability Scan (ZAP Active Scan)

Before running the scan, a login was set up in ZAP so the scan could also test pages that are only visible after logging in (like the SQL Injection and Command Injection pages).

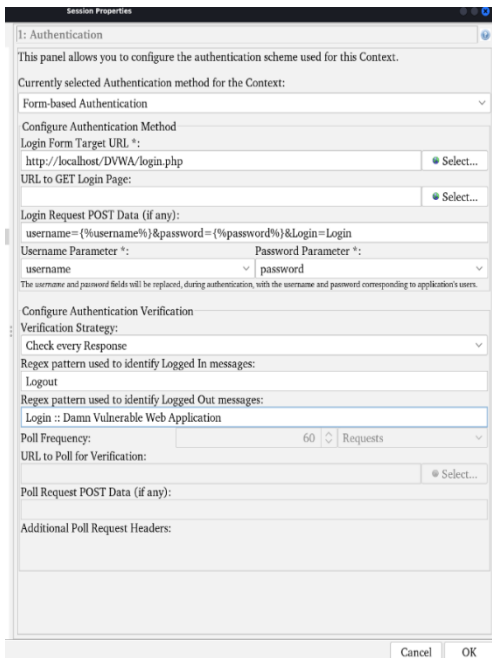


Figure 13 — ZAP authentication set up with the DVWA admin login, so the scan can test pages behind the login page

History Search Filters Output \$Spider \$AMXSpider Active Scan \$ip \$port 20%														
New Scan Name: 1 http://localhost/DVWA/ 20%														
Current Scan: 0 Scan Requests: 1646 New Alerts: 0 Export														
Sent Items Filtered Messages														
ID	Req. Timestamp	Resp. Timestamp	Method	URL	Code	Reason	RTT	Scan Step	Header	Scan Step	Body			
2298	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	POST	http://localhost/DVWA/0wna.php	200	OK	9 ms	327 bytes		1,292 bytes				
2801	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna.js	301	Moved Permanently	3 ms	266 bytes		315 bytes				
2802	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna.php	200	OK	6 ms	327 bytes		1,292 bytes				
2803	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	POST	http://localhost/DVWA/0wna.php	200	OK	6 ms	6,530 bytes		6,530 bytes				
2805	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers	301	Moved Permanently	6 ms	274 bytes		321 bytes				
2807	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers	301	Moved Permanently	6 ms	276 bytes		325 bytes				
2808	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php?v2	404	Not Found	1 ms	217 bytes		217 bytes				
2809	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php?v2/user	404	Not Found	1 ms	216 bytes		217 bytes				
2810	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2user	301	Moved Permanently	1 ms	285 bytes		332 bytes				
2811	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2user	301	Moved Permanently	6 ms	287 bytes		334 bytes				
2812	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/captcha	301	Moved Permanently	4 ms	282 bytes		329 bytes				
2813	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/captcha	301	Moved Permanently	2 ms	287 bytes		334 bytes				
2814	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/captcha.php	301	Moved Permanently	6 ms	282 bytes		329 bytes				
2815	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/c	301	Moved Permanently	3 ms	279 bytes		326 bytes				
2816	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/c	301	Moved Permanently	5 ms	279 bytes		326 bytes				
2817	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/c	301	Moved Permanently	0 ms	277 bytes		324 bytes				
2818	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/jscript.php	301	Moved Permanently	6 ms	282 bytes		332 bytes				
2820	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/user_redirect	301	Moved Permanently	1 ms	288 bytes		335 bytes				
2822	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2	301	Moved Permanently	3 ms	279 bytes		326 bytes				
2823	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload	301	Moved Permanently	6 ms	281 bytes		330 bytes				
2824	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/upload	301	Moved Permanently	6 ms	281 bytes		328 bytes				
2825	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload	301	Moved Permanently	6 ms	282 bytes		329 bytes				
2826	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload	301	Moved Permanently	6 ms	280 bytes		327 bytes				
2828	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload	301	Moved Permanently	6 ms	280 bytes		327 bytes				
2829	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload	301	Moved Permanently	1 ms	280 bytes		327 bytes				
2830	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload	301	Moved Permanently	1 ms	277 bytes		324 bytes				
2833	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload?name=0W...	200	OK	8 ms	327 bytes		4,448 bytes				
2836	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload?name=0W...	200	OK	9 ms	327 bytes		4,448 bytes				
2837	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload?name=0W...	200	OK	14 ms	327 bytes		4,448 bytes				
2840	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload?name=0W...	200	OK	8 ms	327 bytes		4,454 bytes				
2842	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload?name=0W...	200	OK	10 ms	327 bytes		4,479 bytes				
2843	7/16/26, 11:03:51 AM	7/16/26, 11:03:51 AM	GET	http://localhost/DVWA/0wna/headers.php/v2_upload?name=0W...	200	OK	10 ms	327 bytes		4,479 bytes				

Figure 14 — Active Scan running while logged in, thousands of test requests sent automatically

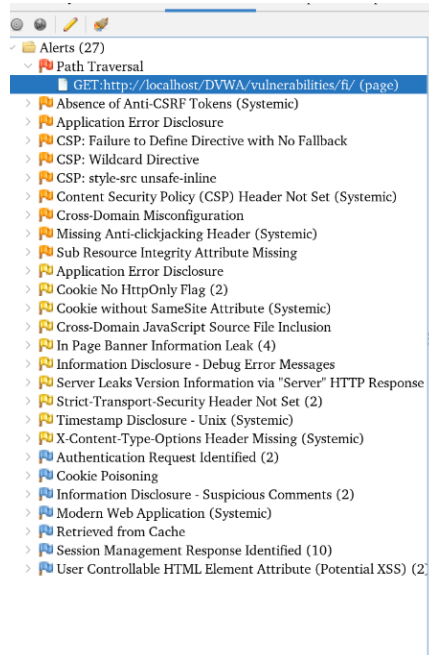


Figure 16 — The most serious finding: a High-risk Path Traversal alert

Figure 15 — Final result: 27 alerts (findings) raised by the scan

5. Findings

Out of the 27 alerts raised by ZAP, plus the Nikto results, the findings below are the ones worth explaining in detail. Many of the remaining alerts are small, repeated versions of the same type of issue (for example, several pages missing the same header), so they are grouped together in the summary table in Section 6.

Finding 1: Path Traversal (File Inclusion Page)

Severity: High

What we did: The scan automatically tried feeding a system file path into the "page" parameter of DVWA's File Inclusion feature.

What we found: The request ``/vulnerabilities/fi/?page=/etc/passwd`` successfully returned the contents of the server's `/etc/passwd` file (the response started with `"root:x:0:0"`, which is the real content of that file on a Linux server).

Why it matters: This means the File Inclusion page does not check what file name it is given. An attacker could use this to read any file on the server that the web application has permission to read, which could include passwords, configuration files, or other private data. This is the most serious finding on this target.

Finding 2: Sensitive Folders Exposed (.git, config, database, tests, docs)

Severity: High

What we did: Nikto scanned the server for folders that should not normally be reachable from outside, and the results were checked by hand.

What we found: The `/.git/`, `/config/`, `/database/`, `/tests/` and `/docs/` folders can all be opened directly in the browser and show a full list of their files.

Why it matters: The `/.git/` folder is the most dangerous of these — if this were a real website, it could let an attacker download the site's entire source code and history, sometimes even old passwords that were once written in the code. The other folders reveal internal setup details and test files that should not be public.

Finding 3: Stored Cross-Site Scripting (XSS)

Severity: Medium

What we did: A script tag was typed into the Stored XSS form to see how DVWA handles it, as a hands-on demonstration of this common vulnerability.

What we found: The script was accepted and saved by the application without being blocked or cleaned, and it was shown back on the page.

Why it matters: Stored XSS means a malicious script typed by one user can end up running in another user's browser later. This is a well-known DVWA training vulnerability, and confirming it here is good practice for recognising this pattern on real applications.

6. Severity Summary — DVWA

#	Finding	Severity	Tool Used
1	Path Traversal (read /etc/passwd via File Inclusion page)	High	OWASP ZAP
2	Sensitive folders exposed (.git, config, database, tests, docs)	High	Nikto (manually confirmed)
3	Stored Cross-Site Scripting (XSS)	Medium	Manual testing
4	Missing security headers (CSP, X-Frame-Options, X-Content-Type-Options)	Medium	OWASP ZAP
5	Missing Anti-CSRF tokens	Medium	OWASP ZAP
6	Cookie flags missing / version & debug info disclosure	Low	Nikto, OWASP ZAP
7	Other repeated low / informational alerts (21 total, e.g. cache, banner info, session handling)	Low / Info	OWASP ZAP

Conclusion for DVWA

DVWA showed clear, confirmed vulnerabilities, which is expected since it is built on purpose to be an easy training target. The most serious issues were the Path Traversal bug and the exposed .git/config/database folders, both of which would let an attacker see private files on a real server. The site is also missing several standard security headers. These are exactly the type of issues a beginner should learn to recognise, since they appear very often in real-world assessments too.

TARGET 2

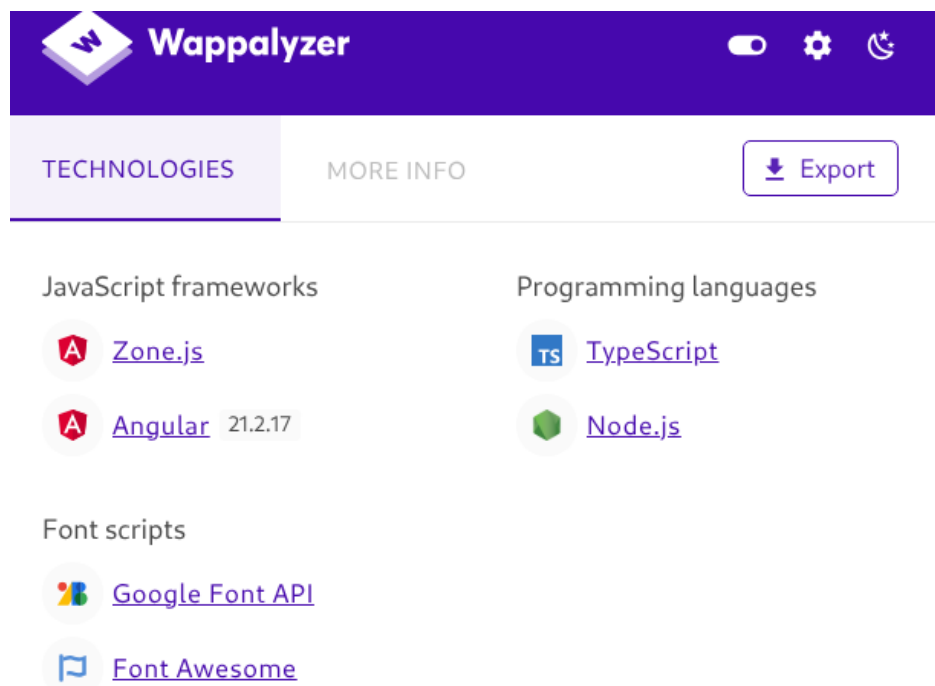
OWASP Juice Shop

7. Target Information

Target	http://localhost:3000
Application Type	Single Page Application (SPA) built with Angular on the frontend and Node.js (Express) on the backend, with a REST API
Testing Type	Black Box, followed by an authenticated scan using a self-registered test account
Note	Juice Shop is intentionally full of hidden security challenges, built by OWASP for learning purposes

8. Reconnaissance

8.1 Technology Check (Wappalyzer & WhatWeb)

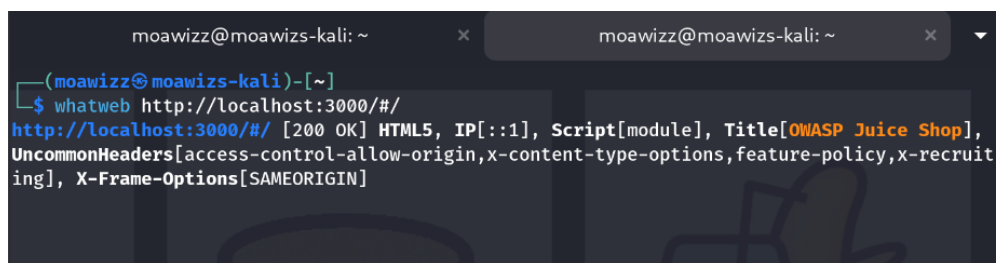


The screenshot shows the Wappalyzer interface with a purple header. Below the header, there are tabs for 'TECHNOLOGIES' and 'MORE INFO', and an 'Export' button. The 'TECHNOLOGIES' tab is active, displaying a list of detected technologies. Under 'JavaScript frameworks', there are 'Zone.js' and 'Angular 21.2.17'. Under 'Programming languages', there are 'TypeScript' and 'Node.js'. Under 'Font scripts', there are 'Google Font API' and 'Font Awesome'.

JavaScript frameworks	Programming languages
Zone.js	TypeScript
Angular 21.2.17	Node.js

Font scripts
Google Font API
Font Awesome

Figure 17 — Wappalyzer result for Juice Shop



```
moawizz@moawizs-kali: ~  
$ whatweb http://localhost:3000/#/  
http://localhost:3000/#/ [200 OK] HTML5, IP[::1], Script[module], Title[OWASP Juice Shop],  
UncommonHeaders[access-control-allow-origin,x-content-type-options,feature-policy,x-recruit  
ing], X-Frame-Options[SAMEORIGIN]
```

Figure 18 — WhatWeb result for Juice Shop

Because Juice Shop is a modern app, its login does not use a simple form submission like DVWA. It sends the username and password as a JSON message to a backend API. This request was captured while exploring manually:

The screenshot shows the Burp Suite interface with the 'Request' tab selected. The 'Header: Text' field is active, displaying the following raw HTTP request:

```
POST http://localhost:3000/rest/user/login HTTP/1.1
host: localhost:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Content-Type: application/json
Content-Length: 46
Origin: http://localhost:3000
Connection: keep-alive
Referer: http://localhost:3000/
Cookie: language=en; welcomebanner_status=dismiss
Priority: u=0

{"email":"' OR '1' = '1' --","password":"'xyz'"}]
```

Figure 20 — The real login request to `/rest/user/login`, sent as JSON data

Figure 21 — Server's response to the login request

Because Juice Shop is built with Angular, a normal Spider cannot fully see its pages. The AJAX Spider, which uses a real browser to crawl, was used instead and worked much better here.

Figure 22 — ZAP Spider results for Juice Shop



Figure 24 — Full list of discovered directories and endpoints, including the /rest/ API folder and an unusual /ftp/ folder

Processed	ID	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Header	Size Resp. Body
	1,620	7/16/26, 11:20:46 AM	GET	http://localhost:3000/assets/public/favicon.js.ico	200	OK	7 ms	456 bytes	15,086 bytes
	1,621	7/16/26, 11:20:47 AM	GET	http://localhost:3000/assets/118a/en.json	200	OK	2 ms	475 bytes	35,465 bytes
	1,622	7/16/26, 11:20:47 AM	GET	http://localhost:3000/assets/118a/en.json	200	OK	6 ms	230 bytes	96 bytes
	1,623	7/16/26, 11:20:48 AM	GET	http://localhost:3000/	304	Not Modified	3 ms	392 bytes	0 bytes
	1,624	7/16/26, 11:20:48 AM	GET	http://localhost:3000/chunks/5K743Z2P.js	304	Not Modified	7 ms	392 bytes	0 bytes
	1,625	7/16/26, 11:20:48 AM	GET	http://localhost:3000/chunks/V53A3LTT.js	304	Not Modified	1 ms	391 bytes	0 bytes
	1,626	7/16/26, 11:20:48 AM	GET	http://localhost:3000/chunks/PX7U3V3.js	304	Not Modified	3 ms	391 bytes	0 bytes
	1,627	7/16/26, 11:20:48 AM	GET	http://localhost:3000/chunks/PX7U3V3.js	304	Not Modified	2 ms	392 bytes	0 bytes
	1,628	7/16/26, 11:20:48 AM	GET	http://localhost:3000/chunks/OKA37MTB.js	304	Not Modified	2 ms	393 bytes	0 bytes
	1,629	7/16/26, 11:20:48 AM	GET	http://localhost:3000/chunks/OKA37MTB.js	304	Not Modified	4 ms	393 bytes	0 bytes
	1,630	7/16/26, 11:20:48 AM	GET	http://localhost:3000/chunks/UYX3ANW4.js	304	Not Modified	3 ms	393 bytes	0 bytes
	1,631	7/16/26, 11:20:48 AM	GET	http://localhost:3000/chunks/QBYXNN7Z.js	304	Not Modified	6 ms	391 bytes	0 bytes
	1,632	7/16/26, 11:20:48 AM	GET	http://localhost:3000/chunks/YVD75JXT.js	304	Not Modified	3 ms	392 bytes	0 bytes
	1,633	7/16/26, 11:20:48 AM	GET	http://localhost:3000/chunks/NYDAMF4.js	304	Not Modified	2 ms	393 bytes	0 bytes
	1,634	7/16/26, 11:20:48 AM	GET	http://localhost:3000/polyfills.js	304	Not Modified	1 ms	392 bytes	0 bytes
	1,635	7/16/26, 11:20:48 AM	GET	http://localhost:3000/scripts.js	304	Not Modified	2 ms	392 bytes	0 bytes
	1,636	7/16/26, 11:20:48 AM	GET	http://localhost:3000/media.js	304	Not Modified	4 ms	393 bytes	0 bytes
	1,637	7/16/26, 11:20:48 AM	GET	http://localhost:3000/styles.css	304	Not Modified	3 ms	393 bytes	0 bytes
	1,638	7/16/26, 11:20:48 AM	GET	https://firefox-settings-attachments.ada.mozilla.org/...	403	Forbidden	0 ms	130 bytes	40 bytes

Figure 23 — AJAX Spider results, showing many more pages and files discovered

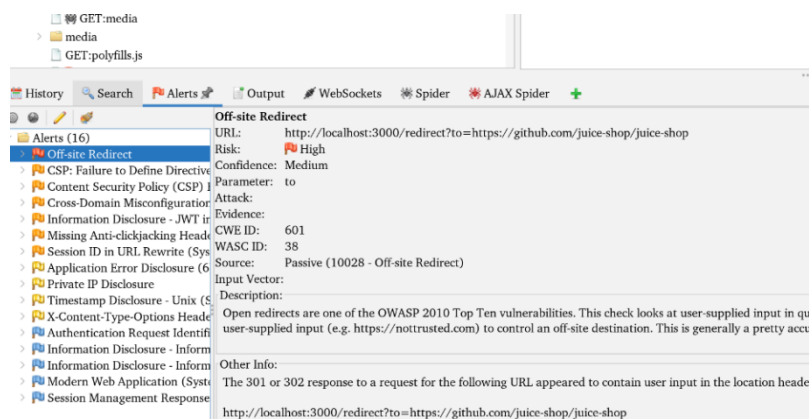


Figure 25 — An alert raised by ZAP while exploring

Two things stood out while mapping the site: a /rest/ folder (the base path for all of Juice Shop's API endpoints) and an /ftp/ folder, which is not something a normal shopping website should openly expose.

9. Automated Scanning

9.1 Server Scan (Nikto)

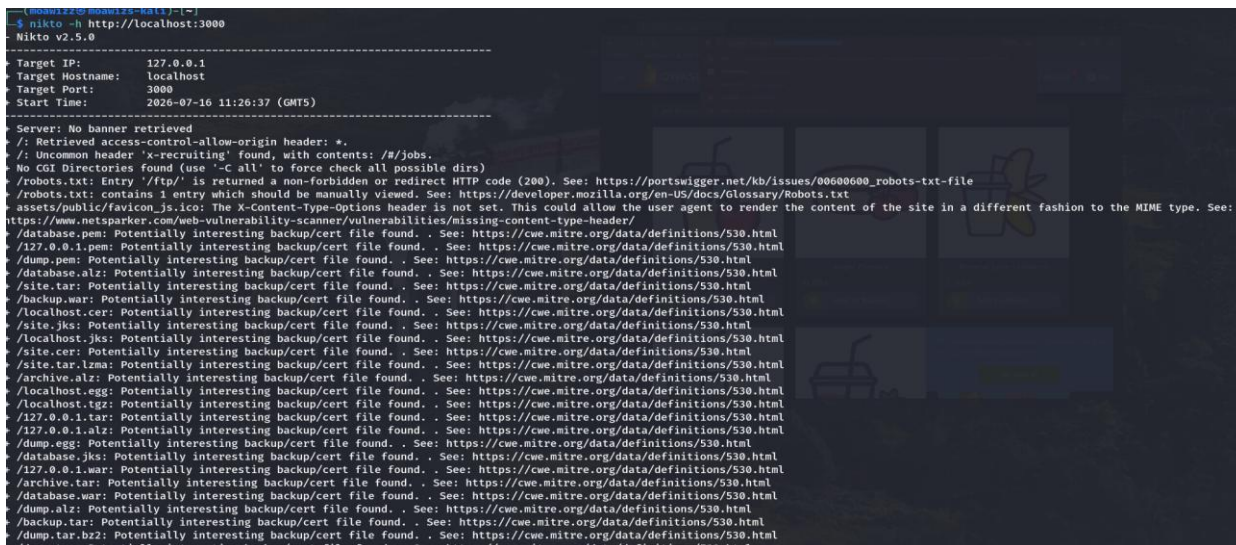


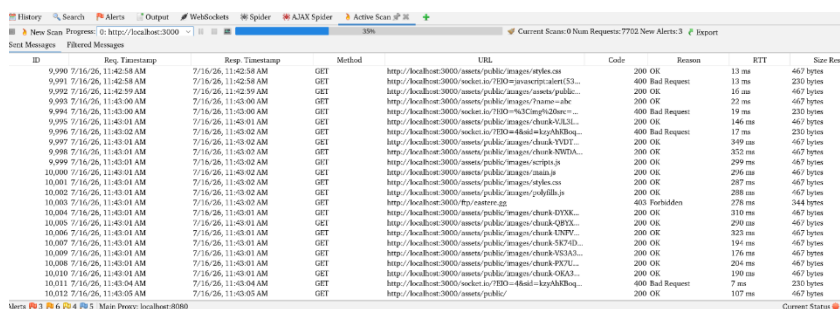
Figure 26 — Nikto scan result for Juice Shop

Nikto reported a long list of "backup/certificate files found" (like database.pem, site.tar, backup.war). These were checked and turned out to be false alarms, not real findings. The reason is that Juice Shop is a Single Page Application: when the server does not recognise a web address, it still replies with "200 OK" and shows the normal app page, instead of a proper "404 Not Found". Nikto reads this "200 OK" as if the guessed file exists, when it actually does not. This is an important lesson: automatic tool results should always be checked before trusting them.

Real findings from Nikto included a custom "x-recruiting" header pointing to a hidden careers page, and a CORS header (access-control-allow-origin: *) that allows any website to make requests to this one — both are minor information disclosure issues, and the /ftp/ folder was also confirmed here through robots.txt.

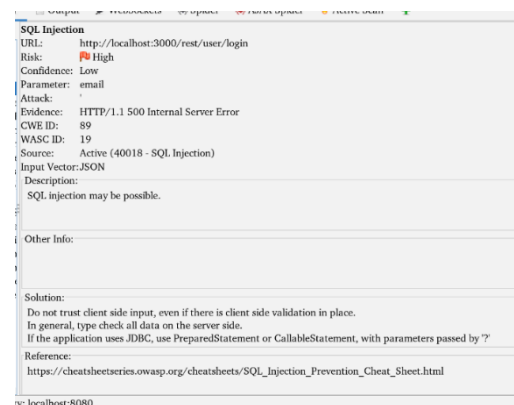
9.2 Automated Vulnerability Scan (ZAP Active Scan)

A test account was registered on Juice Shop and used to set up an authenticated scan, the same way it was done for DVWA, so that pages only visible after logging in could also be tested.



ID	Req. Timestamp	Resp. Timestamp	Method	URL	Code	Reason	RTT	Size Resp
9,990	7/16/26, 11:42:58 AM	7/16/26, 11:42:58 AM	GET	http://localhost:3000/assets/public/images/styles.css	200 OK		13 ms	467 bytes
9,991	7/16/26, 11:42:58 AM	7/16/26, 11:42:58 AM	GET	http://localhost:3000/assets/public/images/styles.css	400 Bad Request		13 ms	230 bytes
9,992	7/16/26, 11:42:59 AM	7/16/26, 11:42:59 AM	GET	http://localhost:3000/assets/public/images/styles.css	200 OK		16 ms	467 bytes
9,993	7/16/26, 11:43:00 AM	7/16/26, 11:43:00 AM	GET	http://localhost:3000/assets/public/images/styles.css	200 OK		22 ms	467 bytes
9,994	7/16/26, 11:43:00 AM	7/16/26, 11:43:00 AM	GET	http://localhost:3000/assets/public/images/styles.css	400 Bad Request		19 ms	230 bytes
9,995	7/16/26, 11:43:01 AM	7/16/26, 11:43:01 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		146 ms	467 bytes
9,996	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	400 Bad Request		17 ms	230 bytes
9,997	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		349 ms	467 bytes
9,998	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		353 ms	467 bytes
9,999	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		299 ms	467 bytes
10,000	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		296 ms	467 bytes
10,001	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		287 ms	467 bytes
10,002	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		288 ms	467 bytes
10,003	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	403 Forbidden		276 ms	344 bytes
10,004	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		310 ms	467 bytes
10,005	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		290 ms	467 bytes
10,006	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		323 ms	467 bytes
10,007	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		194 ms	467 bytes
10,008	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		176 ms	467 bytes
10,009	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		204 ms	467 bytes
10,010	7/16/26, 11:43:02 AM	7/16/26, 11:43:02 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	200 OK		198 ms	467 bytes
10,011	7/16/26, 11:43:04 AM	7/16/26, 11:43:04 AM	GET	http://localhost:3000/assets/public/images/chunk-VJL3L...	400 Bad Request		7 ms	230 bytes
10,012	7/16/26, 11:43:05 AM	7/16/26, 11:43:05 AM	GET	http://localhost:3000/assets/public/	200 OK		107 ms	467 bytes

Figure 27 — Active Scan running against Juice Shop with the test account logged in



SQL Injection

URL: http://localhost:3000/rest/user/login

Risk: **High**

Confidence: Low

Parameter: email

Attack: -

Evidence: HTTP/1.1 500 Internal Server Error

CWE ID: 89

WASC ID: 19

Source: Active (40018 - SQL Injection)

Input Vector: JSON

Description: SQL injection may be possible.

Other Info:

Solution: Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side. If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?'

Reference: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

by: localhost:8080

Figure 28 — One of the High risk alerts raised during the scan

10. Findings

Finding 1: High Risk Alerts SQL Injection

Severity: High

What we did: The ZAP Active Scan sent many automatic test requests to every page and API endpoint of Juice Shop while logged in.

What we found: Two High risk alerts of SQL injection in login page were raised by the scan on Juice Shop's pages/API.

Why it matters: High risk alerts mean the scanner found a strong sign of a real, exploitable weakness (for example, an injection-style issue).

Finding 2: Exposed /ftp/ Folder

Severity: Medium

What we did: This folder was discovered twice — once by the AJAX Spider while mapping the site, and again by Nikto, which found it listed inside the site's own robots.txt file.

What we found: The /ftp/ folder is reachable directly from the browser, and the website's own robots.txt file points to it (robots.txt is meant to tell search engines what NOT to show, but this accidentally reveals it instead).

Why it matters: Folders like this are often used to hide files that should not be public, such as leaked documents. Having it listed in robots.txt makes it very easy for anyone to find, even without deep scanning.

Finding 3: Backend Errors on Login Attempts (Improper Error Handling)

Severity: Medium

What we did: While testing the login API with unexpected input, the server was watched to see how it reacted.

What we found: Some login attempts caused the server to return a "500 Internal Server Error" instead of a normal "invalid login" message.

Why it matters: A well-built application should always reply with a clean, expected error message, never a raw server crash message. A 500 error can sometimes leak details about how the backend code works, and shows the input is not being handled safely.

Finding 4: CORS Misconfiguration & Recruiting Header Leak

Severity: Low

What we did: Nikto and ZAP checked the response headers sent back by the server.

What we found: The server sends "access-control-allow-origin: *" (allowing requests from any website) and a custom "x-recruiting" header pointing to a hidden careers page.

Why it matters: The open CORS header could allow other websites to read data from this site's API in certain situations. The recruiting header is a minor information leak — it reveals a hidden page that was not meant to be advertised this way.

Finding 5: Nikto False Positives (SPA Behaviour)

Severity: Informational

What we did: Nikto guessed many common backup/certificate file names to see if any of them existed on the server.

What we found: Nikto reported many of these as "found", but manual checking showed the server was simply returning its normal homepage with a 200 OK status for every guessed address, instead of a proper 404 Not Found — none of these files actually exist.

11. Severity Summary — Juice Shop

#	Finding	Severity	Tool Used
1	High risk alerts found during authenticated Active Scan	High	OWASP ZAP
2	Exposed /ftp/ folder (also listed in robots.txt)	Medium	AJAX Spider, Nikto
3	Backend errors (500) on some login attempts	Medium	OWASP ZAP
4	CORS wildcard header & x-recruiting header leak	Low	Nikto, OWASP ZAP
5	Nikto false-positive backup/certificate file results (SPA behaviour)	Informational	Nikto (manually verified)

Conclusion for Juice Shop

Juice Shop behaved very differently from DVWA because it is a modern, JavaScript-based application. The AJAX Spider was needed instead of the normal Spider to properly map its pages, and its login uses a JSON API instead of a simple form. Real findings included an exposed /ftp/ folder, backend errors on bad login attempts, and two High risk alerts from the authenticated Active Scan. Some Nikto results turned out to be false alarms caused by how Single Page Applications handle unknown web addresses, which was confirmed by manual checking rather than being reported blindly.

12. Overall Week 1 Conclusion

Both targets were tested using the same five-step process: technology check, technology verification, site mapping, server scan, and automated vulnerability scan. This gave good practice in following a consistent testing method, while also learning to handle two very different types of websites — a classic PHP site (DVWA) and a modern Single Page Application (Juice Shop).

Key lessons learned in Week 1:

- Different applications need different tools — a normal Spider works fine on a simple site like DVWA, but an AJAX Spider is needed for JavaScript-based sites like Juice Shop.
- Automated scanners are very useful, but their results must always be checked by hand. Nikto's backup-file results on Juice Shop looked serious but turned out to be false alarms caused by how the app is built.
- Setting up authentication in ZAP is important, because many real vulnerabilities are hidden behind a login page and will not be found by an unauthenticated scan.
- Both targets had at least one High severity issue confirmed by testing — DVWA's Path Traversal bug and exposed .git folder, and Juice Shop's High-risk alerts from the Active Scan.

This foundation of recon and automated scanning will be built on in the coming weeks with more advanced, manual testing techniques.